

ML Lab - Assignment 2: Linear Regression

Based on ML Lab Assignment 2. The assignment asks for Linear Regression using NumPy with the Normal Equation and Batch Gradient Descent, comparison with scikit-learn, visualizations, and evaluation metrics.

1. Big Picture

Goal: predict median_house_value from the housing features. We build the same linear regression model in three ways: (1) Normal Equation, (2) Batch Gradient Descent, and (3) sklearn LinearRegression. Then compare their performance.

Overall flow: train.csv -> preprocessing -> train models -> predict test.csv -> MSE/RMSE/R2 -> plots -> comparison.

2. Section A: Data Preprocessing

Features must be standardized to zero mean and unit variance, and a bias/intercept column of ones must be added to the NumPy design matrix.

- Separate X (input features) and y (median_house_value).
- Compute training mean (μ) and standard deviation (σ) for every feature.
- Scale: $X_{\text{scaled}} = (X - \mu) / \sigma$.
- Use the training mean and standard deviation to scale the test set.
- For the NumPy models, add a first column of 1s for the intercept.

Why scaling? Features have very different numerical ranges. Scaling makes Gradient Descent more stable and allows one learning rate to work across features.

3. Section B: Normal Equation

This is the closed-form solution. Instead of iteratively learning parameters, the parameters are calculated directly.

$$\theta_{\text{hat}} = (X^T X)^{-1} X^T y$$

How it works: X contains the bias column and standardized features. Compute $X^T X$, its inverse, $X^T y$, and multiply them to obtain theta. Then predict with $y_{\text{hat}} = X \theta$.

Key idea: there is no learning rate or iteration loop. The parameters are obtained directly.

4. Section C: Batch Gradient Descent

Gradient Descent finds theta iteratively by moving in the direction that reduces the loss.

$$\text{Loss: } J(\theta) = (1 / 2n) * \sum ((h_{\theta}(x_i) - y_i)^2)$$

$$\text{Gradient: } \text{grad } J(\theta) = (1 / n) * X^T * (X \theta - y)$$

$$\text{Update: } \theta = \theta - \alpha * \text{grad } J(\theta)$$

- theta: model parameters.
- alpha: learning rate; controls the size of each update.
- iterations: number of updates.
- loss_history: loss stored at each iteration so convergence can be plotted.
- early stopping tolerance: optionally stop when the change in loss becomes very small.

The loss should generally decrease. A learning rate that is too small converges slowly; one that is too large can become unstable.

5. Section E: Scikit-learn Comparison

Fit sklearn's `LinearRegression()` on the same training data and compare its predictions and results with the two NumPy implementations.

Important: sklearn handles the intercept itself, so use the scaled features without manually adding the column of ones. For the NumPy models, the bias column is present.

The sklearn result should be very close to the Normal Equation result because both solve the same linear regression problem. Gradient Descent should also approach them when it converges.

6. Section F: Visualizations

Visualization	What it shows
Loss vs iterations	Whether Gradient Descent is converging.
Different learning rates	How alpha changes convergence speed and stability.
Scaled vs unscaled	Why feature scaling matters for Gradient Descent.
Validation loss	How well the trained model performs on unseen test data.
Loss surface	How loss changes as selected parameters change.

Quick Memory Map

Preprocess -> Standardize -> Add bias -> Solve theta -> Predict -> Measure error -> Visualize -> Compare.